04-01. TVU (Transaction Validation Unit) 아키텍처

개요

TVU (Transaction Validation Unit)는 Solana 검증자에서 다른 노드로부터 받은 블록 데이터를 검증하고 처리하는 핵심 컴포넌트입니다. TPU가 트랜잭션을 처리하여 새로운 블록을 생성하는 역할을 한다면, TVU는 네트워크에서 받은 블록 데이터를 검증하고 로컬 상태에 적용하는 역할을 담당합니다.

TVU 구조체 정의

```
pub  struct Tvu {

fetch_stage: ShredFetchStage,

shred_sigverify: JoinHandle<()>,

retransmit_stage: RetransmitStage,

window_service: WindowService,

cluster_slots_service: ClusterSlotsService,

replay_stage: ReplayStage,

blockstore_cleanup_service: Option<BlockstoreCleanupService>,

cost_update_service: CostUpdateService,

voting_service: VotingService,

warm_quic_cache_service: Option<WarmQuicCacheService>,

drop_bank_service: DropBankService,
```

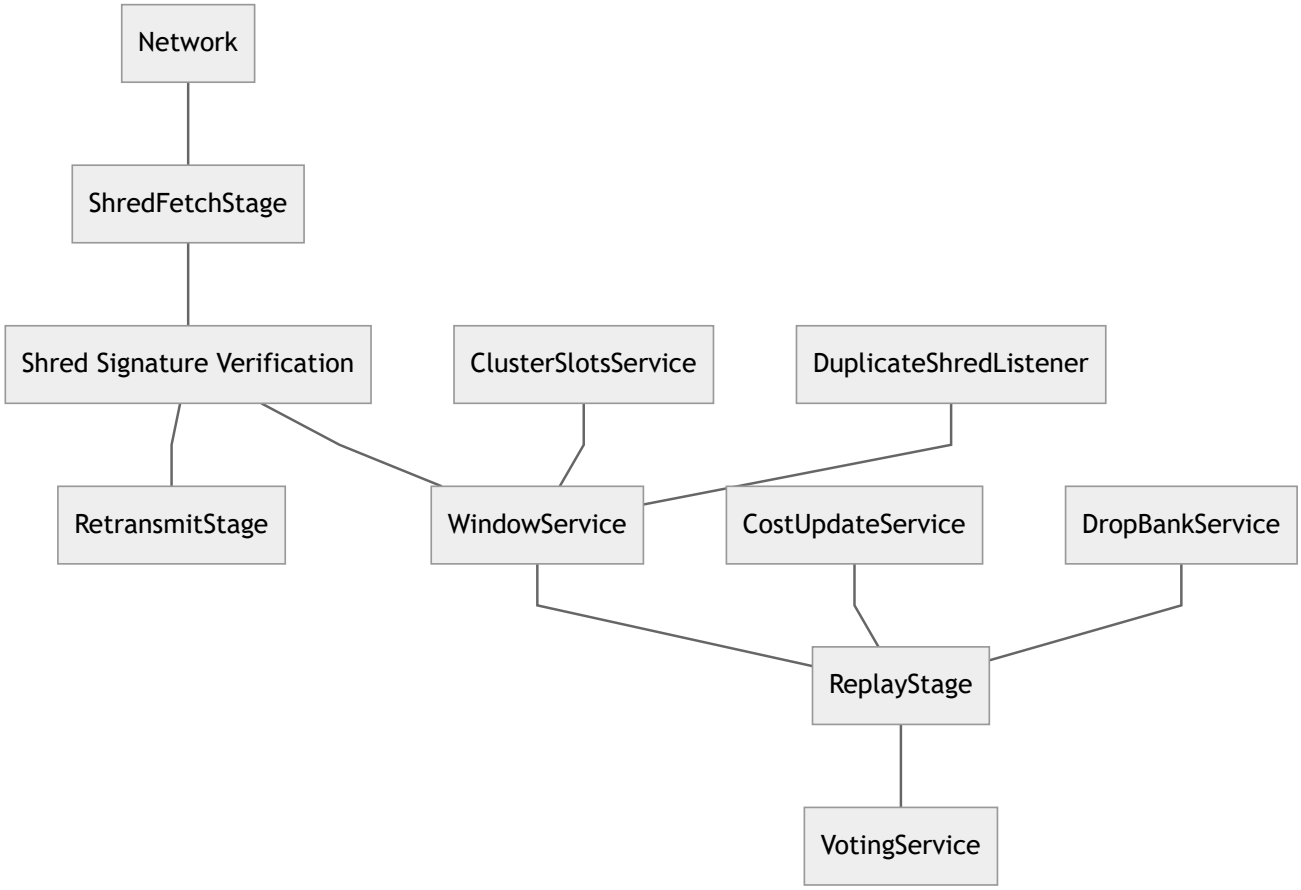
```
duplicate_shred_listener: DuplicateShredListener,

}
```

TVU는 여러 개의 독립적인 서비스들로 구성되어 있으며, 각각이 특정한 역할을 담당합니다.

TVU 검증 파이프라인 아키텍처

TVU의 핵심 검증 파이프라인은 다음과 같은 단계로 구성됩니다:



1. Shred Fetch Stage (Shred 수신 단계)

핵심 역할: 네트워크에서 shred(블록 데이터 조각)를 수신하고 초기 필터링을 수행합니다.

```
let fetch_stage = ShredFetchStage::new(
fetch_sockets,
turbine_quic_endpoint_receiver,
repair_socket.clone(),
repair_quic_endpoint_response_receiver,
fetch_sender,
tvu_config.shred_version,
bank_forks.clone(),
cluster_info.clone(),
turbine_disabled,
exit.clone(),
);
```

- UDP 소켓을 통해 shred 데이터를 수신
- QUIC 프로토콜을 통한 고성능 데이터 전송 지원
- Shred 버전 검증 및 기본적인 유효성 검사
- 수신된 shred를 다음 단계로 전달

2. Shred Signature Verification (서명 검증 단계)

핵심 역할: 수신된 shred의 디지털 서명을 검증합니다.

```
let shred_sigverify = solana_turbine::sigverify_shreds::spawn_shred_sigverify(
cluster_info.clone(),
bank_forks.clone(),
```

```
leader_schedule_cache.clone(),
fetch_receiver,
retransmit_sender.clone(),
verified_sender,
);
```

- 병렬 서명 검증을 통한 성능 최적화
- 리더 스케줄을 기반으로 한 shred 유효성 검증
- 검증된 shred와 재전송이 필요한 shred를 분류

3. Window Service (윈도우 서비스)

핵심 역할: 검증된 shred들을 재조립하여 완전한 블록을 구성합니다.

```
let  window_service = WindowService::new(
blockstore.clone(),
verified_receiver,
retransmit_sender,
repair_socket,
ancestor_hashes_socket,
repair_quic_endpoint_sender,
repair_quic_endpoint_response_sender,
exit.clone(),
repair_info,
leader_schedule_cache.clone(),
```

```
verified_vote_receiver,
completed_data_sets_sender,
duplicate_slots_sender.clone(),
ancestor_hashes_replay_update_receiver,
dumped_slots_receiver,
popular_pruned_forks_sender,
outstanding_repair_requests,
);
```

- Shred 재조립 및 누락된 shred 복구
- Blockstore에 완성된 블록 데이터 저장
- 중복 shred 탐지 및 처리
- Repair 프로토콜을 통한 누락 데이터 요청

4. Replay Stage (재실행 단계)

핵심 역할: 완성된 블록을 재실행하여 상태를 업데이트하고 합의에 참여합니다.

```
let  replay_stage = ReplayStage::new(
replay_stage_config,
blockstore.clone(),
bank_forks.clone(),
cluster_info.clone(),
ledger_signal_receiver,
duplicate_slots_receiver,
```

```
poh_recorder.clone(),
tower,
vote_tracker,
cluster_slots,
retransmit_slots_sender,
ancestor_duplicate_slots_receiver,
replay_vote_sender,
duplicate_confirmed_slots_receiver,
gossip_verified_vote_hash_receiver,
cluster_slots_update_sender,
cost_update_sender,
voting_sender,
drop_bank_sender,
block_metadata_notifier,
log_messages_bytes_limit,
prioritization_fee_cache.clone(),
dumped_slots_sender,
banking_tracer,
popular_pruned_forks_receiver,
)?;
```

- 블록 트랜잭션 재실행 및 상태 검증
- 포크 선택 알고리즘 실행

- Tower BFT 합의 메커니즘 구현
- 투표 생성 및 전송

TVU 소켓 구성

```
pub struct TvuSockets {
pub fetch: Vec<UdpSocket>, // Shred 수신용 소켓들
pub repair: UdpSocket, // Repair 요청/응답용 소켓
pub retransmit: Vec<UdpSocket>, // 재전송용 소켓들
pub ancestor_hashes_requests: UdpSocket, // 조상 해시 요청용 소켓
}
```

TVU 설정 옵션

```
#[derive(Default)]
pub struct TvuConfig {
pub max_ledger_shreds: Option<u64>, // 최대 ledger shred 수
pub shred_version: u16, // Shred 버전
pub repair_validators: Option<HashSet<Pubkey>>, // Repair 요청 대상 검증자들
pub repair_whitelist: Arc<RwLock<HashSet<Pubkey>>>, // Repair 우선순위 검증자들
pub wait_for_vote_to_start_leader: bool, // 리더 시작 전 투표 대기 여부
pub replay_slots_concurrently: bool, // 동시 슬롯 재실행 여부
```

```
}
```

TVU 초기화 과정

`Tvu::new()` 함수는 다음과 같은 순서로 TVU를 초기화합니다:

1. **소켓 설정**: 네트워크 통신을 위한 UDP 소켓들을 Arc로 래핑
2. **채널 생성**: 각 단계 간 데이터 전달을 위한 crossbeam 채널들 생성
3. **Fetch Stage 시작**: Shred 수신을 위한 ShredFetchStage 생성
4. **서명 검증 스레드 시작**: 병렬 서명 검증을 위한 스레드 풀 생성
5. **Retransmit Stage 시작**: Shred 재전송을 위한 RetransmitStage 생성
6. **Window Service 시작**: Shred 재조립을 위한 WindowService 생성
7. **보조 서비스들 시작**: ClusterSlotsService, CostUpdateService 등 생성
8. **Replay Stage 시작**: 블록 재실행을 위한 ReplayStage 생성
9. **기타 서비스들 시작**: VotingService, DropBankService 등 생성

데이터 플로우

TVU 내에서의 데이터 흐름은 다음과 같습니다:

1. **네트워크 → ShredFetchStage**: 원시 shred 데이터 수신
2. **ShredFetchStage → 서명검증**: 수신된 shred 전달
3. **서명검증 → WindowService**: 검증된 shred 전달
4. **서명검증 → RetransmitStage**: 재전송할 shred 전달
5. **WindowService → ReplayStage**: 완성된 블록 데이터 전달
6. **ReplayStage → VotingService**: 투표 결정 전달

7. **ReplayStage → 기타 서비스들**: 상태 업데이트 정보 전달

오류 처리 및 복구

TVU는 다음과 같은 오류 상황들을 처리합니다:

- **네트워크 오류**: 소켓 오류, 타임아웃 등
- **데이터 무결성 오류**: 잘못된 서명, 손상된 shred 등
- **합의 오류**: 포크 충돌, 투표 불일치 등
- **리소스 부족**: 메모리 부족, 디스크 공간 부족 등

각 서비스는 독립적으로 오류를 처리하며, 필요시 상위 레벨로 오류를 전파합니다.

성능 최적화

TVU는 다음과 같은 최적화 기법들을 사용합니다:

- **병렬 처리**: 서명 검증, shred 처리 등을 병렬로 수행
- **비동기 I/O**: QUIC 프로토콜을 통한 비동기 네트워크 통신
- **메모리 풀링**: PacketBatch 재사용을 통한 메모리 할당 최적화
- **배치 처리**: 여러 shred를 한 번에 처리하여 오버헤드 감소

TVU는 Solana의 고성능 블록체인 처리를 가능하게 하는 핵심 컴포넌트로, 복잡한 파이프라인을 통해 안전하고 효율적인 블록 검증을 수행합니다.